Building Multi-Tenant Morpheus Environments with the HPE Terraform Provider

An MSP-focused, practical guide, grounded in a working Coke / Pepsi example.

Morpheus is a hybrid-cloud management platform, and one of its most powerful features is **multi-tenancy**: a single appliance can host many isolated tenants (accounts), each with its own users, roles, clouds, and workloads, all governed by a master tenant. Wiring that up by hand — tenants, permission ceilings, per-tenant admins, clouds, policies — is tedious and error-prone.

This capability maps almost perfectly onto the operating model of a **Managed Service Provider (MSP)**. An MSP runs shared infrastructure on behalf of many customers, and must keep each customer **isolated**, **self-serviceable within guardrails**, **consistently governed**, and **individually billed** — while keeping the provider's own operational overhead low. Morpheus multi-tenancy gives the MSP a *master* (provider) tenant that owns and governs a fleet of *customer* tenants. This post frames the technical patterns in exactly those MSP terms.

We'll walk through how to express a complete multi-tenant Morpheus setup as code using the official **`HPE/hpe`** Terraform provider. Every example is drawn from a real, self-contained configuration that stands up two customer tenants (**Coke** and **Pepsi**) — think of them as two MSP customers — plus a sub-tenant (**Coke-Finance**) *created through* one customer to model that customer's own business unit (created as a flat peer by the provider version used here, though Morpheus 9 itself supports true nested tenancy — see §1), and — per tenant — roles, a bootstrap admin, standard users, an infrastructure group, a VMware cloud, and lifecycle policies.

> **Reading this as an MSP?** The mapping is: **master tenant = the MSP/provider control plane**, **each sub-tenant = a customer** (or a customer's business unit), **multitenant roles = the MSP's standardized service tiers**, **the base role = the contractual guardrail** on what a customer may do, and **Terraform = the MSP's repeatable onboarding pipeline**.

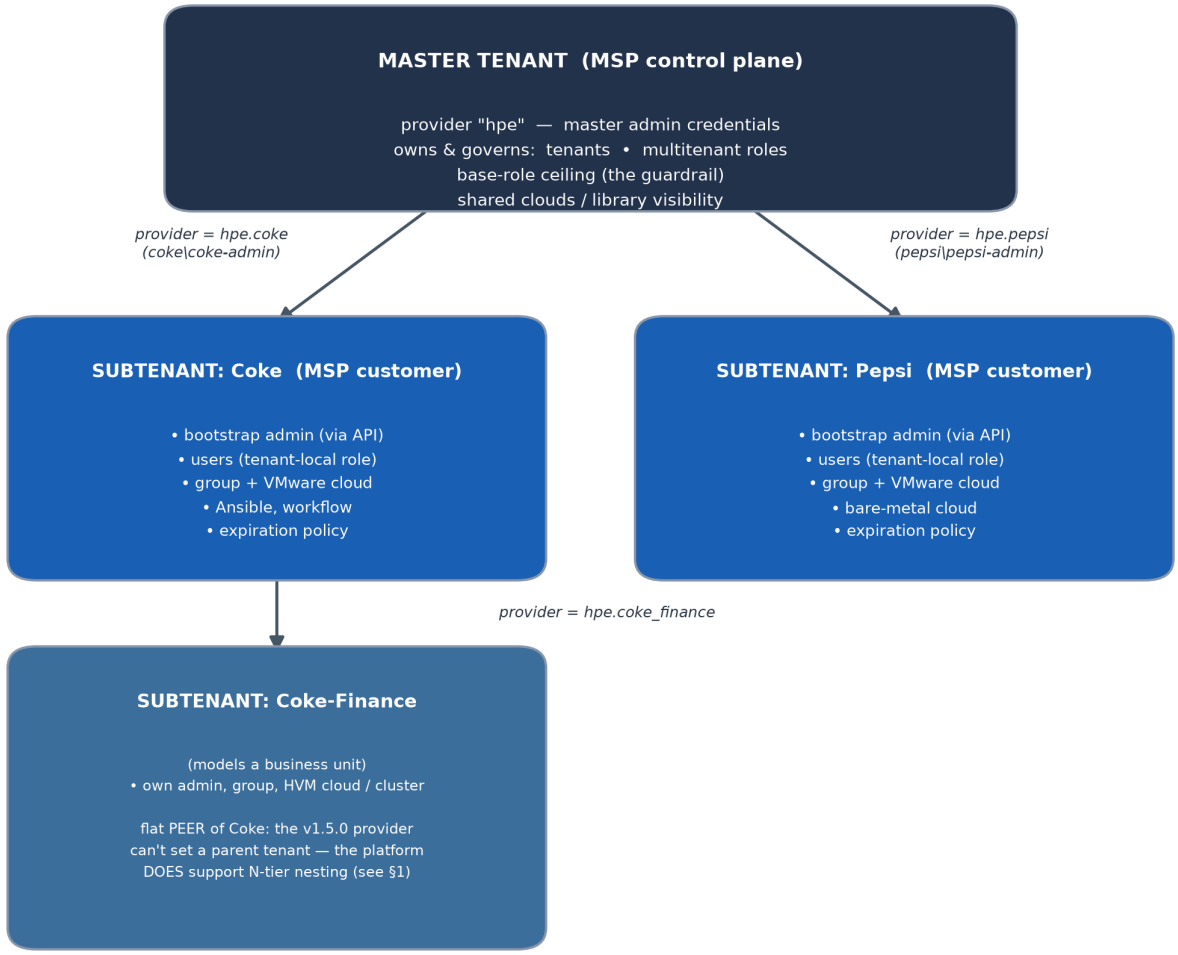
Prerequisites

Before you run any of the examples below, you'll need:

- **Terraform ≥ 1.11.0** — required for the write-only `password_wo` user attribute used throughout (see §2).
- **The `HPE/hpe` provider, pinned to v1.5.0**, and **`hashicorp/external` ≥ 2.3.0** — both are declared in `versions.tf` and fetched by `terraform init`.
- **`curl` and `jq`** on the machine that runs Terraform — the `local-exec` helper scripts (`bootstrap_admin.sh`, `set_inventory_level.sh`, `set_provisioning_settings.sh`, `zone_type_present.sh`) shell out to the Morpheus REST API.
- **Network reachability** from that machine to the Morpheus appliance (`https://<appliance>`), including any self-signed-cert handling (`morpheus_insecure`).
- **A master-tenant admin account** — the master provider authenticates as this identity to create tenants, roles, and bootstrap each customer's first admin.

> **MSP note.** Treat these as the baseline for your onboarding runner. In practice they live in a CI image (Terraform + `curl` + `jq`) with the appliance credentials injected from a secrets manager — see §12.

Architecture at a glance



Everything above the customer boxes lives in the master; everything inside a box is created either by the master provider (`tenant_id`-bearing resources) or that customer's provider alias (resources without a `tenant_id`).

---

1. The core challenge: who creates what?

Morpheus tenancy has two properties that shape the entire design:

1. **Many resources have no `tenant_id` field.** A *group*, an *Ansible integration*, or an *identity source* belongs to whichever tenant the API caller is authenticated as — not to a tenant you name in the resource body.

2. **Sub-tenants are created as flat, top-level peers *by this provider version*.** In the pinned `HPE/hpe` provider **v1.5.0**, the `hpe_morpheus_tenant` resource exposes no parent attribute at all — only `name`, `base_role_id`, `subdomain`, and billing/description fields. So every tenant it creates lands at the top level under the master, and there is no way to express a parent → child relationship through Terraform. That's why "Coke-Finance", though created *through* Coke, is really a **flat peer** of Coke in the master's tenant space, not a child of it — it merely *models* a business unit or reseller-downstream customer.

> **Important — the platform itself is *not* flat on Morpheus 9.** HPE

> Morpheus Enterprise **v8.1.0** introduced *True N-Tier Multi-Tenancy*: a **recursive** hierarchy in which any tenant can be both a consumer of upstream governance and a parent to downstream tenants, to arbitrary depth (Tier-1 master → Tier-2 parent → Tier-3+ business units/customers), with hierarchical RBAC/policy inheritance and cost attribution. So on Morpheus 9, genuinely nested subtenants *are* supported by the product — the flatness here is a limitation of the **provider version**, not of Morpheus. If you need real hierarchy as code today, you must reach for the Morpheus API (e.g. a `local-exec/external` call setting the parent tenant) until the provider exposes it. See References.

The practical consequence: to create objects *inside* a sub-tenant, you must **authenticate as that sub-tenant**. In Terraform terms, that means one **provider alias per tenant**. Objects that *do* accept a `tenant_id` (tenants, clouds, users) can be created by the master provider and simply targeted at the sub-tenant.

So the architecture is:

- A **master provider** — creates tenants, clouds, users, roles.
- One **aliased provider per tenant** — authenticated as that tenant's admin, used for group/integration/identity-source resources that have no `tenant_id`.

In MSP terms: the master provider is the MSP's control plane — the single privileged identity from which every customer is provisioned — while each alias represents the delegated administrative boundary handed to a customer. This is the technical embodiment of the MSP's core promise: *strong isolation between customers, with the provider retaining central control.*

---

2. Pinning the provider

Everything starts in `versions.tf`. The provider is sourced from the public Terraform Registry as **`HPE/hpe`** and pinned deliberately:

```
terraform {
  # >= 1.11 is required for the write-only "password_wo" user attribute.
  required_version = ">= 1.11.0"

  required_providers {
    hpe = {
      source  = "HPE/hpe"
      version = "1.5.0"
    }
    external = {
      source  = "hashicorp/external"
      version = ">= 2.3.0"
    }
  }
}
```

Two things to note:

- **Terraform ≥ 1.11** is required because user passwords use the modern **write-only** attribute (`password_wo`), which never lands in state.
- The **`hashicorp/external`** provider is a companion dependency. It's used to gracefully probe the appliance for optional plugins (more on that later).

> **Tip:** pin the `hpe` provider to an exact version. Several resources behave differently across releases; validate before bumping.

---

3. Configuring the providers

`providers.tf` declares the master provider plus one alias per tenant. The master provider uses straightforward master-admin credentials:

```
# Master-tenant provider. Owns and provisions the Coke/Pepsi sub-tenants.
provider "hpe" {
  morpheus {
    url      = var.morpheus_url
    username = var.morpheus_username
    password = var.morpheus_password
    insecure = var.morpheus_insecure
  }
}
```

Each sub-tenant alias authenticates as *that tenant's* bootstrap admin. The login uses Morpheus's `subdomain\username` convention (a single backslash):

```
provider "hpe" {
  alias = "coke"
  morpheus {
    url      = var.morpheus_url
    username = "coke\\${var.coke_admin_username}"
    password = var.coke_admin_password
    insecure = var.morpheus_insecure
  }
}

provider "hpe" {
  alias = "pepsi"
  morpheus {
    url      = var.morpheus_url
    username = "pepsi\\${var.pepsi_admin_username}"
    password = var.pepsi_admin_password
    insecure = var.morpheus_insecure
  }
}
```

There's real-world nuance captured here. The provider offers a `tenant_subdomain` attribute, but in v1.5.0 it composes the login with a **doubled** backslash (`coke\\coke-admin`), whereas the documented Morpheus API contract expects a single backslash. Embedding the login string directly keeps you aligned with the API contract. Because Morpheus login is **lazy** (per request), these aliases can be *configured* before the admin user actually exists — the resources that use them are deferred to apply time with `depends_on`.

You can chain this further. **Coke-Finance** is a sub-tenant created *through* Coke (using Coke's admin to call the API), and it gets its own alias:

```
provider "hpe" {
  alias = "coke_finance"
  morpheus {
    url      = var.morpheus_url
    username = "coke-finance\\${var.coke_finance_admin_username}"
    password = var.coke_finance_admin_password
    insecure = var.morpheus_insecure
  }
}
```

4. One map to fan out from

The whole configuration is driven from data structures in `locals.tf`. Tenants are a map, so adding a tenant is a single edit that fans out to roles, clouds, policies, and outputs:

```
locals {
  tenants = {
    coke  = { name = "Coke",  subdomain = "coke",  description = "Coke tenant" }
    pepsi = { name = "Pepsi", subdomain = "pepsi", description = "Pepsi tenant" }
  }

  # Tenants created THROUGH the Coke provider (not the master).
  coke_subtenants = {
    coke_finance = {
      name        = "Coke-Finance"
      subdomain   = "coke-finance"
      description = "Tenant created via the Coke tenant provider"
    }
  }
}
```

Tenants themselves are then created with a simple `for_each`:

```
resource "hpe_morpheus_tenant" "this" {
  for_each = local.tenants

  name         = each.value.name
  description  = each.value.description
  subdomain    = each.value.subdomain
  enabled      = true
  base_role_id = hpe_morpheus_role.base.id
  currency     = "USD"
}
```

Note `base_role_id` — every tenant requires a **base role**. That role is far more important than it looks.

Onboarding a new customer, concretely

Because the map drives everything, onboarding a customer is *mostly* data. To add a customer **Acme**, the map-driven parts — its tenant, roles, bootstrap admin, users, and expiration policy — all fan out from a few `locals.tf` entries:

```
# locals.tf
tenants = {
  coke  = { name = "Coke",  subdomain = "coke",  description = "Coke tenant" }
  pepsi = { name = "Pepsi", subdomain = "pepsi", description = "Pepsi tenant" }
  acme  = { name = "Acme",  subdomain = "acme",  description = "Acme tenant" }   # ← new
}

admin_creds = {
  # ...existing...
  acme = { username = var.acme_admin_username, password = var.acme_admin_password }
}

cloud_config = {
  # ...existing...
  acme = {
    name = "Acme VMWare Cloud 1", code = "acmevmwarecloud1"
    api_url = var.acme_cloud_url, datacenter = var.acme_cloud_datacenter
    cluster = var.acme_cloud_cluster
    username = var.acme_cloud_username, password = var.acme_cloud_password
  }
}
```

```
tenant_expiration_days = { coke = 3, pepsi = 4, acme = 5 }        # ← new
tenant_group_ids       = { coke = hpe_morpheus_group.coke.id,
                           pepsi = hpe_morpheus_group.pepsi.id,
                           acme = hpe_morpheus_group.acme.id }    # ← new
```

Two things **can't** be expressed in the map, because Terraform cannot iterate over provider aliases dynamically — so each new customer also needs one provider alias and one group resource (a few lines each):

```
# providers.tf — the customer's delegated admin identity
provider "hpe" {
  alias = "acme"
  morpheus {
    url      = var.morpheus_url
    username = "acme\\${var.acme_admin_username}"
    password = var.acme_admin_password
    insecure = var.morpheus_insecure
  }
}

# clouds.tf — the customer-owned group (created through its own alias)
resource "hpe_morpheus_group" "acme" {
  provider   = hpe.acme
  name       = "Acme Group"
  code       = "acme-group"
  depends_on = [terraform_data.admin]
}
```

Plus the matching `acme_*` variables. That's the honest scope of "add a customer": **map entries drive the bulk automatically; one alias + one group + its variables are the fixed per-customer boilerplate.** It's a small, reviewable pull request — exactly the repeatable unit of work an MSP onboarding pipeline needs.

---

5. Roles, multi-tenancy, and the permission ceiling

This is the subtlest — and most important — part of multi-tenant Morpheus.

Multitenant roles

A role marked `multitenant = true` is **owned by the master** and automatically **copied into every sub-tenant** by Morpheus. Edit it once on the master and the change propagates to every tenant's local copy on a plain `terraform apply`. The example defines two: a `tenant_admin` ("Private Cloud Tenant Owner") and a `tenant_user` ("Private Cloud Tenant Contributor").

```
resource "hpe_morpheus_role" "tenant_admin" {
  name        = "Private Cloud Tenant Owner"
  description = "Shared administrator role for all tenants"
  role_type   = "user"
  multitenant = true

  permissions = {
    default_group_access = "full"
    # ... plus the full feature-permission ceiling (see below)
    feature_permissions  = local.tenant_role_permissions["coke"]
  }
}
```

The base role *is* the ceiling

Here's the trap. The **base (account) role** doubles as the tenant's **permission ceiling**. When a multitenant role is copied into a tenant, the copy's permissions are **masked down** to whatever the base role grants. If the base role grants no feature permissions, every copied role has its feature permissions stripped — and your tenant admin gets HTTP 403 the moment it tries to list roles.

The fix is a deliberately **broad** ceiling defined once and shared by both the base role and the admin role:

```
tenant_ceiling_features = [
  "admin-roles", "admin-users", "admin-groups",
  "admin-accounts",          # manage sub-tenants
  "admin-zones",
  "infrastructure-cluster",  # create/manage clusters
  "admin-cm",                # gates POST /api/integrations
  "tasks", "workflows",
  "admin-containers",        # library instance types
  "app-templates", "apps",
  "provisioning",
  # ... and more Administration features
]
```

There's an important **asymmetry** worth internalizing:

- **Granting** a permission that is *already within* the ceiling to a tenant role propagates with **no recreate**.
- **Raising** the ceiling (adding a new code) is applied only when roles are *seeded* into a tenant — it is **NOT retroactive**. An existing tenant must be recreated to pick up a newly added ceiling code.

The takeaway: **keep the ceiling wide up front** to avoid painful tenant recreations later. Per-tenant extras (e.g., Coke gets the full `provisioning-*` family) layer cleanly on top:

```
tenant_role_permissions = {
  for tenant in keys(local.tenants) : tenant => concat(
    local.tenant_ceiling_permissions,
    [for code in lookup(local.tenant_extra_feature_codes, tenant, []) :
      { code = code, access = "full" }],
  )
}
```

Why this matters to an MSP. The base-role ceiling *is* the contractual guardrail. It is where the MSP encodes "what this customer is allowed to do to themselves" — the boundary between customer self-service and provider-only operations. The per-tenant extras map is how the MSP expresses **service tiers** or **contract add-ons**: a premium customer (here, Coke) receives the full provisioning feature set, while a standard customer (Pepsi) gets the baseline. Because the whole thing is data-driven, offering a new tier is a config change, not a bespoke build — exactly the leverage an MSP needs to serve many customers without per-customer engineering.

---

6. The bootstrap-admin problem (and an elegant workaround)

Sub-tenant *users* need a `role_id`. But you cannot assign a multitenant master role's id directly through `hpe_morpheus_user`: Morpheus swaps in the sub-tenant's **local copy** of the role at apply time, and the provider's post-apply consistency check then fails with *"planned … does not correlate with any element in actual."* (This is a known provider bug — the resource echoes back the local role id for a `Required` field.)

There's also a chicken-and-egg problem: the alias provider that could resolve local role ids must authenticate *as* a user that doesn't exist yet.

The example solves this in two layers:

1. The first (bootstrap) admin is created straight through the Morpheus API via a `local-exec` provisioner, bypassing Terraform's consistency check entirely:

```
resource "terraform_data" "admin" {
  for_each = local.tenants

  triggers_replace = {
    tenant_id = hpe_morpheus_tenant.this[each.key].id
    username  = local.admin_creds[each.key].username
    role_id   = hpe_morpheus_role.tenant_admin.id
  }

  provisioner "local-exec" {
    command = "bash \"${path.module}/bootstrap_admin.sh\""
    environment = {
      MORPH_URL  = var.morpheus_url
      TENANT_ID  = tostring(hpe_morpheus_tenant.this[each.key].id)
      ADMIN_USER = local.admin_creds[each.key].username
      ROLE_ID    = tostring(hpe_morpheus_role.tenant_admin.id)
      # ... credentials passed via environment, not argv
    }
  }
}
```

The `bootstrap_admin.sh` helper is idempotent and security-conscious: it feeds the password to `curl` on **stdin** and passes the bearer token from a `0600` temp file, so **no secret ever appears in the process list** (`argv`). It also checks whether the admin already exists before creating it, so re-runs and recovery from failed applies are safe no-ops. (This requires `curl` and `jq` on the machine running Terraform.)

2. Standard users are then created by the master provider, but assigned the tenant-**local** role id resolved by a data source scoped to the sub-tenant (authenticated as the now-existing bootstrap admin):

```
data "hpe_morpheus_role" "coke_user_role" {
  provider = hpe.coke
  name     = hpe_morpheus_role.tenant_user.name
  depends_on = [terraform_data.admin, hpe_morpheus_role.tenant_user]
}

resource "hpe_morpheus_user" "coke_user" {
  count                = var.coke_user_count
  tenant_id            = hpe_morpheus_tenant.this["coke"].id
  username             = "coke_user${count.index}"
  password_wo          = var.user_password
  password_wo_version = 1
  role_ids             = [data.hpe_morpheus_role.coke_user_role.id]
}
```

Because the resolved id matches what Morpheus stores, `planned == actual` and the consistency check passes. Note the **write-only** `password_wo` / `password_wo_version` pair — the password is applied but never persisted to state.

---

7. Groups and clouds: authenticating as the right tenant

`hpe_morpheus_group` has **no** `tenant_id` — so a group belongs to whichever tenant its provider

is logged in as. Each group is therefore created through the matching alias:

```
resource "hpe_morpheus_group" "coke" {
  provider = hpe.coke
  name     = "Coke Group"
  code     = "coke-group"
  depends_on = [terraform_data.admin]   # wait for the admin to exist
}
```

Clouds, by contrast, **do** take a `tenant_id`, so they're created by the master provider and targeted at the sub-tenant, with `group_id` pointing at the tenant-owned group:

```
resource "hpe_morpheus_cloud" "vmware" {
  for_each = local.cloud_config

  name      = each.value.name
  tenant_id = hpe_morpheus_tenant.this[each.key].id
  group_id  = local.tenant_group_ids[each.key]

  cloud_type_code  = "vmware"
  visibility       = "private"
  data_center_name = each.value.datacenter   # set explicitly (provider bug workaround)

  config = {
    apiUrl     = each.value.api_url
    datacenter = each.value.datacenter
    cluster    = each.value.cluster
    username   = each.value.username
    password   = each.value.password
    enableVnc  = "on"
  }
}
```

This resource captures two useful lessons about mature IaC against a real API:

- `data_center_name` is set **explicitly** to dodge a provider bug where an unset value returns `""` after apply, causing spurious "inconsistent result" errors on later updates.
- The generic `config` map is passed through to the Morpheus API **verbatim**, so you use the API's own key names (e.g. `enableVnc = "on"`). Unknown keys are silently ignored — convenient, but a source of quiet mistakes.

Gracefully probing for optional plugins

Not every appliance has every cloud type installed. Rather than letting the plan hard-fail, the example uses the **`hashicorp/external`** provider to probe `/api/zone-types` and gate a resource on the result:

```
data "external" "baremetal_cloud_type" {
  program = ["bash", "${path.module}/zone_type_present.sh"]
  query = {
    url  = var.morpheus_url
    code = "hpe-baremetal-plugin.cloud"
    # ...
  }
}

resource "hpe_morpheus_cloud" "pepsi_baremetal" {
  count = data.external.baremetal_cloud_type.result.present == "true" ? 1 : 0
  # ... HPE bare-metal (BMaaS) cloud, created only when the plugin exists
}
```

This pattern — a tiny `external` data source returning `{"present":"true"}` — is how you keep a single configuration portable across appliances with different tech packs installed.

8. Everything else fans out too

Because the tenant map is the single source of truth, the remaining resources follow the same `for_each` shape:

- **Lifecycle policies** (`policies.tf`) — a fixed-expiration policy per tenant group, driven by `local.tenant_expiration_days`.
- **Tasks & workflows** (`tasks.tf`, `workflows.tf`) — a shell task per tenant; Coke additionally gets a provisioning workflow that runs an Ansible task.
- **Integrations** (`integrations.tf`) — a Coke Ansible (git) integration, created through `hpe.coke` because, like a group, it has no `tenant_id`.
- **Identity sources** (`identity_sources.tf`) — an optional Active Directory source for Coke.

MSP angle: these are the levers of day-two service delivery. Per-tenant **lifecycle policies** enforce cost-control and hygiene SLAs (e.g. auto-expiring lab instances) uniformly across the customer base; **tasks/workflows** and **integrations** let the MSP ship standardized automation into each customer; and per-tenant **identity sources** let each customer **bring their own directory** (their own AD/LDAP/SSO) so end users authenticate against the customer's own IdP rather than provider-managed accounts — a common contractual requirement.

---

9. When Terraform isn't enough: the escape hatch

A recurring theme in this configuration is the disciplined use of `local-exec`

- the Morpheus REST API to paper over provider gaps. The example documents **four workarounds for provider bugs** and **three for provider limitations**, including:

- `bootstrap_admin.sh` — create a tenant's first admin (multitenant `role_ids` bug).
- `set_inventory_level.sh` — PUT the **top-level** `zone.inventoryLevel` that the provider only writes into nested config.
- `set_provisioning_settings.sh` — apply cloud-init settings the provider resource errors on.
- `zone_type_present.sh` — the `external` probe shown above.

Each script is idempotent and authenticates as the master admin. Keeping these narrow, well-documented, and idempotent is what makes the difference between a fragile hack and a maintainable escape hatch.

For an MSP, this discipline is not optional. A provider onboarding hundreds of customers cannot tolerate a fragile pipeline: **idempotent, credential-safe, recovery-tolerant** automation is what lets onboarding run unattended and be re-run safely after a partial failure — the difference between a repeatable service and a manual runbook.

---

10. Known limitations & provider caveats

Being honest about where the provider fights you is part of running this in production. The reference configuration carries **four workarounds for confirmed provider bugs** and **three**

for provider limitations — every one of them documented, with reproductions, in `TF/bugs/README.md`. Here is the symptom → workaround map so you know what you're signing up for on **v1.5.0**:

#	Symptom on v1.5.0	Root cause	Workaround
1	"Inventory existing instances" never takes effect	Provider writes the level only into nested `config`; Morpheus honours the **top-level** `zone.inventoryLevel`	`set_inventory_level.sh` (PUTs the top-level field)
2	`setting_provisioning` fails a *successful* apply	Provider expects an echoed settings object; Morpheus returns only `{"success":true}`	`set_provisioning_settings.sh`
3	Multitenant `role_ids` → "inconsistent result"	Morpheus swaps the master role id for a tenant-local one on write	Bootstrap admin via API; scoped `role` data source
4	Console / generic config keys don't stick on update	Generic `config` map drops unknown keys and all but an allowlist on update	Set `enableVnc` on create; or `config_vmware` block
5	`cloud_type` hard-errors when the type is absent	SDK throws instead of returning empty	`external` probe (`zone_type_present.sh`)
6	No way to look up a cluster **layout** id	No cluster-layout data source (only cluster *types*)	Pass `coke_hvm_layout_id` (looked up via API)
7	`node_type` errors on duplicate names	Data source filters on name/id only	Pass the node-type id directly

Two further **gaps** (not bugs, just unimplemented in v1.5.0) shape the design:

- **No `api` persona** in `persona_permissions` (only `standard`, `serviceCatalog`, `vdi`), so "enable API access for all users" can't be expressed through the provider in this version.
- **`hpe_morpheus_tenant` has no `parent` attribute**, which is exactly why Coke-Finance is a flat peer rather than a true nested sub-tenant (see §1).

The common thread: each gap is bridged with the **same narrow, idempotent `local-exec` + API pattern** described next, and every workaround is pinned to a provider version so a future upgrade can retire it deliberately.

> **MSP note.** Track these against provider releases. Each workaround is technical debt with a clear payoff condition — when `HPE/hpe` closes the bug, you delete a script and simplify the pipeline. `TF/bugs/README.md` is the ledger.

---

11. Running it

```
terraform init
terraform plan
terraform apply
```

Everything is designed to converge in a **single apply**: sub-tenant data sources are deferred to apply time and depend on the bootstrap admins, so the whole graph — tenants → roles → admins → tenant-scoped groups/clouds/users — resolves in one pass. Just make sure `curl` and `jq` are on the machine running Terraform, since the workarounds shell out to the API.

For an MSP, the single-apply property is the onboarding SLA in code: adding a new customer is a pull request that adds the entries shown above, reviewed and applied through the same GitOps pipeline as everything else. Customer onboarding becomes **auditable, repeatable, and fast**.

12. Operating this in production

The reference configuration is deliberately self-contained, but an MSP running it for real needs to make a few operational decisions the example leaves open.

Validation & change review

Wire the basics into CI before the fleet grows: `terraform fmt -check`, `terraform validate`, and a **reviewed `terraform plan`** on every pull request. One caveat specific to this design: the `local-exec` helpers mean the plan is **not fully declarative** — the API-side effects of `terraform_data`/`local-exec` resources don't appear as normal resource diffs. Reviewers should treat a change to any helper script (or the variables it consumes) as a change that *will* hit the appliance, even when `plan` shows little. Pin the provider (§2) so plans stay reproducible, and keep the `TF/bugs/README.md` ledger (§10) up to date so reviewers know which diffs are workarounds versus intent.

State

Local state does not survive a team, a CI runner, or a laptop. Use a **remote backend** (S3 + DynamoDB lock, Terraform Cloud, GitLab, etc.) so state is shared, locked, and versioned:

```
terraform {
  backend "s3" {
    bucket         = "msp-morpheus-tfstate"
    key            = "morpheus/prod.tfstate"
    region         = "eu-west-1"
    dynamodb_table = "msp-morpheus-tflock"
    encrypt        = true
  }
}
```

A design choice worth making early: **one monolithic state for all customers** (simplest; every apply touches the whole fleet) versus **state-per-customer** (via workspaces or separate root modules — blast radius is limited to one customer, at the cost of more moving parts). Most MSPs graduate to state-per-customer as the fleet grows, so a change for one customer can never break another.

Secrets

Every `*_admin_password`, cloud credential, and iLO password is a secret. The module keeps them out of state where the provider allows (user passwords use the write-only `password_wo`), and out of process argv in the helper scripts — but **you** must keep them out of `terraform.tfvars` in version control. Options, in increasing order of robustness:

- **Environment variables** — `export TF_VAR_acme_admin_password=…` (never committed).
- **A secrets manager** — pull from HashiCorp Vault (the `vault` provider), AWS Secrets Manager, or your CI's secret store at plan time.

The shipped `.gitignore` already excludes `terraform.tfvars`, `*.tfstate`, and `.terraform/`; treat that as the floor, not the ceiling.

Quotas and capacity limits

RBAC controls *what* a customer can do; it does **not** cap *how much* they can consume. For true

MSP isolation, pair the permission ceiling with Morpheus **tenant resource quotas / plans** (max instances, vCPU, memory, storage per tenant) so one customer cannot exhaust shared capacity. These are set on the tenant in Morpheus; check your provider version for coverage and fall back to the API (the same `local-exec` pattern used elsewhere) if the resource does not yet expose them.

---

13. Mapping to MSP requirements

Pulling the threads together, here is how the technical patterns above satisfy the classic requirements an MSP must meet:

MSP requirement	How Morpheus + this Terraform delivers it
Tenant isolation	Each customer is a separate Morpheus tenant with its own users, roles, groups and clouds. Resources without a `tenant_id` are created *only* through that customer's provider alias, so they cannot leak across tenants.
Centralized governance / control plane	The master tenant owns tenants, multitenant roles and the base-role ceiling. The MSP edits policy once on the master and Morpheus propagates it to every customer.
Standardized service tiers	Multitenant roles (`tenant_admin`, `tenant_user`) are the MSP's catalog of standardized roles; the per-tenant `tenant_extra_feature_codes` map expresses tier/add-on differences (premium Coke vs standard Pepsi) as data.
Guardrailed self-service	The base role acts as a permission *ceiling* — customers administer themselves, but only within the boundary the MSP contractually sets.
Fast, repeatable onboarding	Adding a customer is a single entry in `local.tenants`; roles, admin, clouds, policies and users all fan out. The whole graph converges in one `terraform apply`.
Delegated administration	Each customer gets a bootstrap admin and can manage its own users/roles/clouds via its alias, without provider involvement for routine tasks.
Bring-your-own identity	Per-tenant identity sources (e.g. Active Directory) let each customer authenticate against its own IdP.
Chargeback / showback	Clouds are created with `costing_mode = "costing"`, and lifecycle policies (per-tenant expirations) enforce cost hygiene — the inputs an MSP needs for per-customer billing and consumption reporting.
Hierarchical / reseller tenancy	Morpheus 9 (Enterprise v8.1.0+) natively supports **True N-Tier Multi-Tenancy** — recursive parent→child tenants with inherited RBAC/policy and hierarchical cost attribution. The Coke-Finance pattern models this today, but as a **flat peer**, because the pinned provider v1.5.0 can't set a parent tenant; expressing real hierarchy currently needs the Morpheus API (see §1).
Auditability & compliance	Everything is declarative code in version control: every tenant, role and permission change is a reviewed, traceable commit — the audit trail MSPs need for compliance.
Operational resilience at scale	Idempotent, credential-safe `local-exec` helpers make onboarding re-runnable and recovery-tolerant across a large customer fleet.

The key insight for an MSP evaluating Morpheus: the platform's tenancy model lines up cleanly with the provider/customer boundary, and the `HPE/hpe` Terraform provider turns that model into a **repeatable, GitOps-driven onboarding pipeline** — the operational backbone of a managed service.

Key takeaways

1. **One provider alias per tenant.** Resources without a `tenant_id` belong to whoever is authenticated; those with one can be master-created and targeted. *For an MSP, this alias is the customer's administrative boundary.*

2. **The base role is the permission ceiling** — the MSP's contractual guardrail. Keep it *broad* up front; raising it later is not retroactive and forces tenant recreation.
3. **Multitenant roles are your service catalog.** They propagate automatically, but you cannot assign their master id to a sub-tenant user directly — resolve the tenant-local copy via a scoped data source.
4. **Bootstrap the first admin via the API**, then let Terraform take over — this is what makes unattended customer onboarding possible.
5. **Drive everything from one map** so onboarding a new customer is a one-line, reviewable change.
6. **Probe, don't assume** — use the `external` provider to stay portable across appliances with different plugins/tech packs.
7. **Model tiers and add-ons as data** (`tenant_extra_feature_codes`) so new service offerings are config, not code.

Wrapping up

Multi-tenant Morpheus and the MSP operating model fit together almost one-to-one: the master tenant is your control plane, each sub-tenant is a customer, multitenant roles are your service catalog, and the base role is the contractual guardrail. Expressed through the `HPE/hpe` provider, that model becomes a **GitOps onboarding pipeline** — a new customer is a reviewed pull request that converges in a single `terraform apply`.

The honest caveat is the provider itself: on **v1.5.0** you'll lean on a handful of narrow, idempotent `local-exec` + API workarounds (§10), and true nested tenancy still needs the API even though the platform supports it (§1). None of that undermines the approach — it just means treating those workarounds as tracked, retireable debt. Start from the two-tenant reference here, add a remote backend, a secrets manager, and CI validation, and you have the operational backbone of a managed service.

References

Providers used

- **HPE Terraform Provider (`HPE/hpe`)**, pinned to **v1.5.0** — Terraform Registry: https://registry.terraform.io/providers/HPE/hpe/latest · Docs: https://registry.terraform.io/providers/HPE/hpe/latest/docs · Source: https://github.com/HPE/terraform-provider-hpe
- **`hashicorp/external`**, >= **2.3.0** — used for the optional cloud-type probe: https://registry.terraform.io/providers/hashicorp/external/latest/docs

Platform & APIs

- **HPE Morpheus** (platform): https://www.hpe.com/ · Morpheus documentation: https://docs.morpheusdata.com/ · Morpheus REST API reference: https://apidocs.morpheusdata.com/ — the source of truth for the endpoints the `local-exec` helpers call (`/oauth/token`, `/api/users`, `/api/zone-types`, `/api/provisioning-settings`, etc.).
- **HPE Morpheus Enterprise — *True N-Tier Multi-Tenancy*** (introduced in **v8.1.0**; the definitive statement that Morpheus 9 supports recursive nested tenants): HPE Community blog, F. Escobar, https://community.hpe.com/t5/the-cloud-experience-

everywhere/introducing-true-n-tier-multi-tenancy-in-hpe-morpheus-enterprise/ba-p/7263068 · v8.1.0 release documentation: https://support.hpe.com/hpesc/public/docDisplay?docId=sd00007510en_us&page=GUID-709AAADB-A9C1-40B6-AD22-958EE7E6F312.html

- **`HPE/hpe` provider v1.5.0 — `hpe_morpheus_tenant` schema** (shows no parent attribute, hence the flat-peer behaviour here): https://registry.terraform.io/providers/HPE/hpe/1.5.0/docs/resources/morpheus_tenant

External repositories referenced by the configuration

- The Coke Ansible integration (`integrations.tf`) points at a **public Morpheus sample Ansible repository** by default (via `var.coke_ansible_url`, branch `master`); swap in your own git repo and add authentication for private use.

Provider bug/limitation ledger

- **`TF/bugs/README.md`** (in this repository) — the authoritative, reproducible record of the four provider bugs and three limitations summarized in §10, with root-cause detail and the exact workaround file for each.

Resources & data sources featured

`hpe_morpheus_tenant`, `hpe_morpheus_role`, `hpe_morpheus_user`, `hpe_morpheus_group`, `hpe_morpheus_cloud`, `hpe_morpheus_integration_ansible`, the `hpe_morpheus_role` data source, and the `external` data source — see the provider docs above for the full schema of each.

This walkthrough is based on a self-contained two-tenant (Coke / Pepsi) reference configuration; the same patterns scale to any number of tenants.

> **Documentation note (tenancy hierarchy).** Morpheus 9 is *not* limited to flat tenancy. HPE Morpheus Enterprise Software **v8.1.0** introduced *True N-Tier Multi-Tenancy* — a recursive model in which *"any tenant can function as both a consumer of upstream governance and a provider of downstream control,"* enabling *"hierarchical delegation … to arbitrary depth"* (Tier-1 master → Tier-2 parent → Tier-3+), with hierarchical RBAC/policy inheritance and cost attribution. The flat-peer behaviour in this walkthrough is a constraint of the pinned **`HPE/hpe` provider v1.5.0**, whose `hpe_morpheus_tenant` resource exposes no parent attribute — not a limitation of the Morpheus platform. To build genuine hierarchy as code today, set the parent tenant via the Morpheus API (the same `local-exec/external` escape hatch used elsewhere here) until the provider supports it. Sources: HPE Community, *Introducing True N-Tier Multi-Tenancy in HPE Morpheus Enterprise* (https://community.hpe.com/t5/the-cloud-experience-everywhere/introducing-true-n-tier-multi-tenancy-in-hpe-morpheus-enterprise/ba-p/7263068); `HPE/hpe` provider v1.5.0 `hpe_morpheus_tenant` schema.